

Università degli Studi di Urbino Carlo Bo

Informatica Scienza e Tecnologia

Anno Accademico 2025/2026

**Progetto d'esame di Reti di Calcolatori Analisi delle
vulnerabilità di un sito o applicazione web realizzata con
React + ExpressJS**

Andrea Pedini

Matricola n. 322918

1. Obiettivo del Progetto

L'obiettivo del progetto è quello di effettuare un'analisi delle principali vulnerabilità e rischi a cui si è soggetti quando si va a sviluppare un sito web o una web app "manualmente", utilizzando librerie o framework front-end come React, e un back-end realizzato in ambiente Node.js, ad esempio con framework minimali come Express.js.

Andremo perciò a valutare diverse tipologie di attacchi a cui sono suscettibili tali applicazioni e siti web valutando i vari componenti dei sistemi: dal lato Client, ossia quando l'utente ha ottenuto la propria pagina, al lato Server, ossia il codice in esecuzione sulla macchina remota che mantiene le risorse utilizzate dal sito.

Per fare ciò seguiremo le linee guida di OWASP Top 10.

OWASP Top 10 è un documento standard di consapevolezza per sviluppatori e sicurezza web, che elenca i 10 rischi di sicurezza più critici per le applicazioni.

2. Vulnerabilità principali

Secondo OWASP, le principali aree di vulnerabilità e tipologie di attacchi a cui sono suscettibili gli applicativi web, facendo riferimento a OWASP Top 10:2025, sono:

- A01 - Broken Access Control
- A02 - Security Misconfiguration
- A03 - Software Supply Chain Failures
- A04 - Cryptographic Failures
- A05 - Injection
- A06 - Insecure Design
- A07 - Authentication Failures
- A08 - Software or Data Integrity Failures
- A09 - Security Logging & Alerting Failures
- A10 - Mishandling of Exceptional Condition

Andiamo ora ad analizzare ognuna di queste vulnerabilità singolarmente e in modo dettagliato.

3. A01:2025 — Broken Access Control

La vulnerabilità di tipo **Broken Access Control** si verifica quando il sistema non applica correttamente le restrizioni sugli accessi alle risorse.

In un'applicazione React + ExpressJS questo problema è particolarmente comune quando il backend non verifica correttamente l'identità dell'utente che effettua una richiesta.

Una delle vulnerabilità più diffuse è l'**IDOR (Insecure Direct Object Reference)**.

Backend vulnerabile

```
app.get('/api/users/:id', async (req, res) => {  
  
  const user = await db.get(  
    'SELECT * FROM users WHERE id = ?',  
    [req.params.id]  
  );  
  
  res.json(user);  
});
```

Problema

Un utente autenticato può modificare manualmente l'ID nella richiesta HTTP:

```
GET /api/users/2
```

ottenendo così informazioni appartenenti ad altri utenti.

Questo permette:

- accesso non autorizzato ai dati
- furto di informazioni sensibili
- violazione della privacy
- escalation dei privilegi

Fix

È necessario verificare che l'utente autenticato possa realmente accedere alla risorsa richiesta.

```
app.get('/api/users/:id', authenticate, async (req, res) => {  
  if (req.user.id !== Number(req.params.id)) {  
    return res.status(403).json({  
      error: 'Forbidden'  
    });  
  }  
  
  const user = await db.get(  
    'SELECT * FROM users WHERE id = ?',  
    [req.user.id]  
  );  
  
  res.json(user);  
});
```

4. A02:2025 — Security Misconfiguration

Le vulnerabilità di configurazione rappresentano errori derivanti da configurazioni errate di server, framework, middleware o servizi.

Stack Trace Esposti

Codice vulnerabile

```
app.use((err, req, res, next) => {  
  res.status(500).send(err.stack);  
});
```

Problema

L'applicazione espone direttamente:

- stack trace
- path interni del server
- librerie utilizzate
- dettagli tecnici dell'ambiente

Queste informazioni possono aiutare un attaccante a preparare attacchi mirati.

CORS troppo permissivo

Codice vulnerabile

```
app.use(cors({  
  origin: '*',  
  credentials: true  
}));
```

Problema

Qualsiasi dominio può effettuare richieste al server.

Questo aumenta enormemente il rischio di:

- furto dati
- CSRF
- utilizzo illecito delle API

Fix

```
app.use(cors({  
  origin: 'https://myapp.com',  
  credentials: true  
}));
```

È buona pratica consentire solamente i domini autorizzati.

5. A03:2025 — Software Supply Chain Failures

Questa categoria riguarda vulnerabilità introdotte da dipendenze esterne, librerie compromesse o componenti software non sicuri.

Dipendenza compromessa

```
{
  "dependencies": {
    "random-package-123": "^1.0.0"
  }
}
```

Problema

Pacchetti malevoli possono:

- rubare variabili d'ambiente
- eseguire codice arbitrario
- installare backdoor
- compromettere il server

Uso di versioni vulnerabili

```
{
  "dependencies": {
    "lodash": "4.17.11"
  }
}
```

Versioni obsolete possono contenere CVE pubbliche già note.

Fix

```
npm audit  
npm audit fix
```

Aggiornare regolarmente le dipendenze:

```
{  
  "dependencies": {  
    "lodash": "^4.17.21"  
  }  
}
```

È inoltre importante:

- utilizzare lockfile (`package-lock.json`)
- verificare la provenienza delle librerie
- evitare pacchetti poco conosciuti

6. A04:2025 — Cryptographic Failures

Le vulnerabilità crittografiche derivano dall'uso scorretto della cifratura o dalla gestione errata delle credenziali.

Password salvate in chiaro

Backend vulnerabile

```
app.post('/register', async (req, res) => {  
  await db.run(  
    'INSERT INTO users(username, password) VALUES (?, ?)',  
    [req.body.username, req.body.password]  
  );  
  res.send('ok');  
});
```

Problema

Se il database viene compromesso, tutte le password risultano immediatamente leggibili.

Fix con bcrypt

```
const bcrypt = require('bcrypt');

app.post('/register', async (req, res) => {

  const hash = await bcrypt.hash(req.body.password, 12);

  await db.run(
    'INSERT INTO users(username, password) VALUES (?, ?)',
    [req.body.username, hash]
  );

  res.send('ok');
});
```

Le password devono sempre essere:

- hashate
- salate
- mai salvate in chiaro

JWT Secret Hardcoded

Codice vulnerabile

```
const token = jwt.sign(user, 'secret123');
```

Problema

Un attaccante che ottiene il codice sorgente può falsificare token JWT validi.

Fix

```
const token = jwt.sign(user, process.env.JWT_SECRET);
```

Le chiavi segrete devono essere conservate in:

- variabili d'ambiente
- secret manager
- sistemi protetti

7. A05:2025 — Injection (SQL Injection)

Le vulnerabilità di Injection permettono a un attaccante di eseguire codice o query arbitrarie.

Backend vulnerabile

```
app.post('/login', async (req, res) => {  
  
  const query =  
    `SELECT * FROM users  
     WHERE username='${req.body.username}'  
     AND password='${req.body.password}'`;  
  
  const user = await db.get(query);  
  
  res.json(user);  
});
```

Problema

L'input dell'utente viene concatenato direttamente nella query SQL.

Attacco

Input:

```
' OR '1'='1
```

Query risultante:

```
SELECT * FROM users  
WHERE username='' OR '1'='1'
```

L'attaccante può bypassare l'autenticazione.

Fix con query parametrizzate

```
const user = await db.get(  
  'SELECT * FROM users WHERE username=? AND password=?',  
  [req.body.username, req.body.password]  
);
```

Le query parametrizzate impediscono l'interpretazione dell'input come codice SQL.

8. XSS — Cross Site Scripting

Gli attacchi XSS permettono l'esecuzione di JavaScript malevolo nel browser della vittima.

React vulnerabile

```
function Comment({ text }) {  
  return (  
    <div  
      dangerouslySetInnerHTML={{  
        __html: text  
      }}  
    />  
  );  
}
```

Payload

```
<script>alert('XSS')</script>
```

Problema

L'attaccante può:

- rubare cookie
- rubare JWT
- modificare il DOM
- effettuare azioni a nome dell'utente

Fix

```
function Comment({ text }) {  
  return <div>{text}</div>;  
}
```

oppure utilizzare sanitizzazione:

```
npm install dompurify
```

```
import DOMPurify from 'dompurify';  
  
<div  
  dangerouslySetInnerHTML={{  
    __html: DOMPurify.sanitize(text)  
  }}  
</div>
```

9. CSRF — Cross Site Request Forgery

Le vulnerabilità CSRF permettono a un sito malevolo di effettuare richieste sfruttando la sessione autenticata dell'utente.

Backend vulnerabile

```
app.post('/api/change-email', (req, res) => {  
    updateEmail(req.body.email);  
    res.send('updated');  
});
```

Attacco

```
<form action="https://victim.com/api/change-email" method="POST">  
  <input name="email" value="attacker@mail.com">  
</form>  
  
<script>  
document.forms[0].submit();  
</script>
```

Problema

Il browser invia automaticamente i cookie di sessione.

Fix con CSRF Token

```
npm install csrf
```

```
const csrf = require('csrf');  
  
app.use(csrf());  
  
app.get('/form', (req, res) => {  
  res.json({  
    csrfToken: req.csrfToken()  
  });  
});
```

È inoltre consigliato usare:

- cookie `SameSite`
- token anti-CSRF
- verifica dell'header Origin

10. A06:2025 — Insecure Design

Questa categoria riguarda errori progettuali nell'architettura dell'applicazione.

Business Logic Flaw

Backend vulnerabile

```
app.post('/api/purchase', async (req, res) => {  
  
  const item = await db.get(  
    'SELECT * FROM items WHERE id=?',  
    [req.body.itemId]  
  );  
  
  const finalPrice = req.body.price;  
  
  chargeUser(finalPrice);  
  
  res.send('ok');  
});
```

Problema

L'utente può inviare:

```
{  
  "itemId": 1,  
  "price": 1  
}
```

e acquistare un prodotto costoso a prezzo alterato.

Fix

```
const finalPrice = item.price;
```

Mai fidarsi dei dati provenienti dal client.

11. A07:2025 — Authentication Failures

Questa categoria comprende errori relativi all'autenticazione e gestione delle sessioni.

Session ID prevedibile

```
const sessionId = username + Date.now();
```

Problema

Gli identificativi di sessione possono essere indovinati.

Nessun Rate Limiting

```
app.post('/login', loginHandler);
```

Problema

Permette attacchi brute-force sulle password.

Fix

```
npm install express-rate-limit
```

```
const rateLimit = require('express-rate-limit');

app.use('/login', rateLimit({
  windowMs: 15 * 60 * 1000,
  max: 5
}));
```

È inoltre consigliato implementare:

- MFA
- password policy robuste
- timeout di sessione

12. A08:2025 — Software or Data Integrity Failures

Questa categoria riguarda problemi di integrità del software e dei dati.

Deserializzazione insicura

```
const data = JSON.parse(req.body.payload);
eval(data.code);
```

Problema

L'utilizzo di `eval()` può portare a:

- Remote Code Execution (RCE)
- compromissione completa del server

Auto Deploy senza verifica

```
deploy:
  script:
    - git pull
    - npm install
    - pm2 restart app
```

Problema

Codice compromesso potrebbe essere distribuito automaticamente in produzione.

Fix

Utilizzare:

- firma digitale degli artifact
- pipeline CI/CD sicure
- verifica dell'integrità
- code review

Evitare sempre `eval()`.

13. A09:2025 — Security Logging & Alerting Failures

Questa vulnerabilità riguarda logging e monitoraggio insufficienti.

Nessun logging

```
app.post('/login', async (req, res) => {  
    const user = await login(req.body);  
    res.json(user);  
});
```

Problema

Tentativi di brute-force o attività sospette non vengono rilevati.

Logging di password

```
console.log(req.body);
```

Problema

Le credenziali finiscono nei log.

Fix

```
logger.warn({  
    event: 'failed_login',  
    username: req.body.username,  
    ip: req.ip  
});
```

I log devono:

- evitare dati sensibili
- essere monitorati
- generare alert automatici

14. A10:2025 — Mishandling of Exceptional Conditions

Questa categoria riguarda la gestione errata delle eccezioni.

Backend vulnerabile

```
app.get('/api/data', async (req, res) => {  
    const data = JSON.parse(req.query.payload);  
    res.json(data);  
});
```

Problema

Input non validi possono causare crash dell'applicazione.

Esempio:

```
?payload={
```

Fix

```
app.get('/api/data', async (req, res) => {  
  try {  
    const data = JSON.parse(req.query.payload);  
    res.json(data);  
  } catch (err) {  
    res.status(400).json({  
      error: 'Invalid payload'  
    });  
  }  
});
```

Una corretta gestione delle eccezioni previene:

- denial of service
- crash del server
- perdita di disponibilità

15. Conclusioni

Le applicazioni web moderne sviluppate con React ed ExpressJS offrono grande flessibilità e rapidità di sviluppo, ma espongono anche numerose superfici d'attacco se non vengono adottate adeguate pratiche di sicurezza.

L'analisi effettuata mostra come molte vulnerabilità derivino non tanto dai framework stessi, quanto da:

- errori di configurazione
- progettazione insicura
- gestione errata degli input
- dipendenze vulnerabili
- autenticazione debole

L'utilizzo delle linee guida OWASP Top 10 rappresenta uno strumento fondamentale per sviluppare applicazioni più sicure e resilienti.

È quindi essenziale adottare:

- validazione degli input
- gestione sicura delle sessioni
- controllo degli accessi
- crittografia adeguata
- monitoraggio e logging
- aggiornamento costante delle dipendenze

La sicurezza applicativa deve essere considerata fin dalle prime fasi dello sviluppo ("Security by Design"), e non solamente come intervento correttivo successivo.